Ciphers

Security Report

Authenticated file encryption with a post-quantum hybrid KEM

Version 1.0.3

Author: Jean-François Lachance-Caumartin

Document: Security architecture & threat model

Status: Public

June 14, 2026

Contents

1 Overview

Ciphers is a small GTK3 desktop application for Linux that encrypts and decrypts arbitrary files under a single user password. It provides modern authenticated encryption (AEAD) and an optional *post-quantum hybrid key encapsulation* layer that combines CRYSTALS-Kyber-1024 with the X448 elliptic curve.

This document describes the cryptographic design, the on-disk file format, the memory-hardening measures, the threat model, and — equally important — the properties that Ciphers explicitly does *not* provide. It is intended for users evaluating the tool for sensitive data and for reviewers auditing its guarantees.

1.1 Design goals

- Confidentiality and integrity of file contents under a password.
- Optional protection against a future quantum adversary ("harvest-now, decrypt-later").
- Resistance to tampering, reordering and truncation of ciphertext.
- Keeping secrets (password, derived keys, plaintext) off disk.
- A small, auditable codebase built on vetted libraries.

1.2 Non-goals

- Hiding the existence, size or type of an encrypted file (no steganography or plausible deniability).
- Filename, metadata or directory-structure encryption.
- Multi-recipient / public-key sharing workflows.
- Protection against a compromised host (keylogger, malicious GTK input method, cold-boot attack on live RAM).

2 Cryptographic primitives

All symmetric primitives are provided by **libsodium**; the X448 curve is provided by **OpenSSL** (`libcrypto`); the Kyber-1024 implementation is the CRYSTALS reference code (`KYBER_K=4`, SHAKE/FIPS-202 variant), with randomness drawn from libsodium. Argon2id is provided by **libargon2**.

Purpose	Primitive	Parameters
AEAD (default)	AES-256-GCM	256-bit key, 96-bit nonce, 128-bit tag
AEAD (alt.)	XChaCha20-Poly1305	256-bit key, 192-bit nonce, 128-bit tag
AEAD (alt.)	ChaCha20-Poly1305 (IETF)	256-bit key, 96-bit nonce, 128-bit tag
Key derivation	Argon2id	see Table 1
PQ KEM	CRYSTALS-Kyber-1024	NIST security level 5
Classical KEM	X448 ECDH	448-bit curve
Hybrid combiner	BLAKE2b (keyed)	domain "`HYBRID`", 32-byte output
Key wrap	XChaCha20-Poly1305	wraps the hybrid secret key

2.1 Key derivation

The password is stretched with Argon2id. Three presets are offered; the parameters are stored in the file header so decryption reproduces them.

Preset	Memory	Iterations	Parallelism
Basic	256 MiB	3	4 lanes
Medium †	1 GiB	3	4 lanes
Strong	4 GiB	4	8 lanes

Table 1: Argon2id presets. †Medium is the minimum recommended for sensitive data.

A fresh 16-byte random salt is generated per file, so identical passwords on different files yield independent keys and the KDF output cannot be precomputed or shared across files.

3 File format

Every encrypted file begins with a fixed 40-byte header:

Offset	Size	Field
0	8	Magic `"CIPHERS\0"`
8	1	Format version (1 = password-only, 2 = hybrid)
9	1	Cipher id
10	1	KDF id (1 = Argon2id)
11	1	KDF level (informational)
12	4	Argon2 iterations (little-endian)
16	4	Argon2 memory in KiB (little-endian)
20	4	Argon2 parallelism (little-endian)
24	16	Salt

In hybrid files (version 2) a *hybrid block* follows the header, containing the wrap nonce, the wrapped hybrid secret key and the KEM ciphertext. The base AEAD nonce follows, then the payload frames.

3.1 Chunked authenticated streaming

The plaintext is split into 64 KiB chunks. Each chunk is encrypted as an independent AEAD frame:

$$\text{frame} = \underbrace{\texttt{len}}_{4\,\text{B}} \;\|\; \underbrace{\text{ciphertext} \,\|\, \text{tag}}_{\texttt{len}\,\text{B}}.$$

The per-chunk nonce is the random base nonce with a 64-bit little-endian counter XORed into its trailing eight bytes, guaranteeing a unique nonce per chunk under a given key. The associated data bound into each frame is

$$\text{AD} = \underbrace{\texttt{counter}}_{8\,\text{B}} \;\|\; \underbrace{\texttt{final\_flag}}_{1\,\text{B}}.$$

Binding the counter authenticates chunk *ordering*; binding the end-of-stream flag authenticates that the last chunk really is the last one. Together these detect:

- **Tampering** — any modified ciphertext byte fails its Poly1305 / GCM tag.

- **Reordering** — swapping frames breaks the counter binding.
- **Truncation** — dropping the final frame(s) leaves a chunk whose `final_flag` no longer matches; the missing-final-block check also fires.
- **Splicing / extension** — appending frames after the authenticated final frame is rejected.

These properties were validated empirically with round-trip, bit-flip, truncation and frame-swap tests across all three ciphers and the hybrid mode.

4 Post-quantum hybrid KEM

When hybrid mode is enabled the file's AEAD key is not derived directly from the password. Instead:

1. A fresh Kyber-1024 + X448 keypair is generated per file.
2. The hybrid *secret* key (`kyber_sk` ‖ `x448_sk`) is wrapped with XChaCha20-Poly1305 under a master key derived from the password via Argon2id, bound to the domain string `"CIPHERS-HYBRID-WRAP"`.
3. The KEM encapsulates to the matching public key, producing a 32-byte shared secret that becomes the AEAD file key.
4. The shared secret is the keyed BLAKE2b combination of the Kyber and X448 secrets, so the key stays secure as long as *either* primitive holds.

On decryption the password unwraps the secret key, which decapsulates the KEM ciphertext back to the AEAD key. A single password therefore unlocks the file while the construction remains secure against a quantum adversary who can break X448 but not Kyber (and vice versa).

Why public keys are not stored. Decapsulation needs only the secret key and the KEM ciphertext, so the per-file public keys are deliberately omitted. This removes ∼1.6 KiB of otherwise unauthenticated header bytes: the wrap nonce and wrapped secret key are covered by the wrap's Poly1305 tag, and the KEM ciphertext is covered transitively — altering it changes the decapsulated key and fails every subsequent frame tag.

5 Memory hardening

- **No swap.** Passwords, derived keys and plaintext buffers are pinned with `sodium_mlock`, which prevents them from being paged to the swap file and marks them `MADV_DONTDUMP`.
- **Zeroed after use.** The corresponding `sodium_munlock` wipes each buffer before releasing it.
- **No core dumps.** `RLIMIT_CORE` is set to zero and, on Linux, `PR_SET_DUMPABLE` is cleared (blocking `ptrace` and `/proc`-based core capture).
- **Guarded password field.** The password `GtkEntry` is backed by a custom `GtkEntryBuffer` whose text lives in `sodium_malloc`'d guarded memory (guard pages, locked, zeroed on free).

Residual exposure. GTK itself may keep transient copies of typed text in ordinary heap memory for on-screen rendering (Pango), the clipboard and the input method. These copies are outside Ciphers' control. The hardening therefore *reduces* but cannot fully eliminate in-memory exposure of the password.

6 Threat model

6.1 In scope — what Ciphers defends against

- A passive adversary who obtains the ciphertext file and attempts to recover the plaintext without the password.
- An active adversary who modifies, reorders, truncates or extends the ciphertext: all such tampering is detected and decryption aborts.
- Offline password guessing, slowed by memory-hard Argon2id and a per-file salt.
- A future quantum adversary (hybrid mode), under the "harvest-now, decrypt-later" assumption.
- Recovery of secrets from the swap file or a core dump on a non-compromised host.

6.2 Out of scope — what Ciphers does *not* protect against

- **A compromised host.** Keyloggers, malicious input methods, memory scrapers running as the same or a more privileged user, or cold-boot attacks on live RAM are out of scope.
- **Weak passwords.** Argon2id slows guessing but cannot rescue a low-entropy password. Strength is the user's responsibility.
- **Metadata leakage.** The file's existence, exact size (to within one 64 KiB chunk plus per-frame overhead), and the chosen cipher/KDF parameters are visible in the clear. Filenames are not encrypted.
- **Deniability.** The `"CIPHERS\0"` magic makes an encrypted file trivially identifiable as such.
- **Long-term key escrow / recovery.** There is no backdoor and no recovery mechanism; a forgotten password means permanent data loss.

7 Security properties at a glance

Property	Provided
Confidentiality of file contents	**Yes**
Integrity / tamper detection (AEAD)	**Yes**
Reordering detection	**Yes**
Truncation & extension detection	**Yes**
Per-file unique salt and nonce	**Yes**
Memory-hard password stretching (Argon2id)	**Yes**
Post-quantum confidentiality (hybrid mode)	**Yes**
Secrets kept off swap / out of core dumps	**Yes**
Authenticated KDF & format parameters	**Yes**(transitively)
Filename / metadata encryption	**No**

Property	Provided
Hiding that a file is encrypted (deniability)	**No**
Protection on a compromised host	**No**
Password recovery / escrow	**No**
Multi-recipient public-key sharing	**No**

8 Hardening notes & known limitations

- **Untrusted headers are validated.** On decryption the Argon2id parameters read from the file are range-checked (iterations, parallelism, and the rule $m\_cost \geq 8 \times parallelism$, capped at 4 GiB / 16 iterations / 16 lanes) so a malicious file cannot force an out-of-memory allocation or a hang.
- **Atomic output.** Output is written to a sibling temporary file and `rename()`d into place only on success, so a wrong password or a cancelled run never truncates a pre-existing output. The same-file check covers the input, the output and the temporary file.
- **AES-256-GCM availability.** AES-256-GCM requires hardware AES support; on CPUs without it the cipher is unavailable and the ChaCha-based AEADs should be used instead.
- **No formal audit.** The codebase has not undergone an independent third-party security audit. It relies on well-reviewed libraries (libsodium, OpenSSL, libargon2) and the CRYSTALS-Kyber reference implementation, but the integration code itself is unaudited.

Recent security fixes (v1.0.3)

- Removed a use-after-free in the GUI progress timer that could fire against freed state if the window was closed immediately after a job finished.
- Prevented the output's temporary file from resolving to, and truncating, the input file.
- Tightened validation of Argon2id parameters from untrusted headers.
- Replaced silent truncation of over-long paths with an explicit error.

9 Recommendations for users

1. Use a high-entropy passphrase; the KDF cannot compensate for a weak one.
2. Choose **Medium** or **Strong** key strength for sensitive data.
3. Enable **hybrid mode** for data that must stay confidential for years (post-quantum margin).
4. Run Ciphers only on a trusted, up-to-date system; it cannot defend a compromised host.
5. Keep an independent, secure backup of the password — there is no recovery path.